

Learn Low-Level Design in 25 Days: Day 17 — Designing a Cache System (LRU, LFU, Write-Through)

Ever Felt Your System Getting Slower the More It Runs?

TechPreneur

Published October 30, 2025

Free: No

Read on Freedium · original

Contents

Why Caching Exists	0
Types of Cache Use Cases:	0
Designing a Cache System: The Big Picture	0
1. Eviction Policies — Deciding What to Forget	0
Least Recently Used (LRU)	0
Least Frequently Used (LFU)	0
2. Write Policies — Keeping Cache & DB in Sync	0
Write-Through Cache	0
Write-Back (Lazy Write)	0
Write-Around	0
3. Pulling It All Together	0
Example: User Profile Cache	0
Visual Snapshot	0
Try It Yourself	0
Summary	0
What's Next	0

Imagine you're building a social feed for an app like Instagram. The same "top 20 posts" query runs every second for millions of users. Your database is crying. The latency spikes, queries queue up, and suddenly your system feels… sluggish.

That's when every senior engineer says the magic word: **"We need a cache."**

Welcome to **Day 17 of "Learn Low-Level Design in 25 Days"** — where we'll dive deep into designing a **Cache System**, the secret performance booster behind almost every large-scale system — from Netflix's recommendations to Twitter timelines.



By the end of this post, you'll understand **how caching works**, **how to implement LRU/LFU eviction**, and **how write strategies (write-through, write-back)** affect consistency and reliability.

(If you're new here, you can follow the full series here: Low-Level Design in 25 Days (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8).)

Why Caching Exists

A cache is simply **a temporary storage layer** that holds frequently accessed data in memory for fast retrieval.

If your database is a library, your cache is the librarian's desk — holding the most requested books for instant access.

Without caching:

- Every request hits the database.
- Latency increases.
- Scalability suffers.

With caching:

- You serve hot data in milliseconds.
- The database load drops.
- Your system feels "instant."

Types of Cache Use Cases:

- **Application Cache**: In-memory caching (e.g., user sessions, configs)
- **Distributed Cache**: Shared across servers (e.g., Redis, Memcached)
- **Database Cache**: Query result caching
- **CDN Cache**: Edge content caching for images, videos, etc.

Designing a Cache System: The Big Picture

Let's sketch what a simple caching layer might look like:

```
Client ---> Cache ---> Database
          |      \
        Hit     Miss ---> Load data + store in cache
```

When data is requested:

1. The cache checks if it has the value (`cache hit`).
2. If not, it fetches from the database (`cache miss`), stores it, and returns it.

Sounds easy, right? The real art lies in **what to evict** when the cache is full, and **how to sync with the database** when data changes.

1. Eviction Policies — Deciding What to Forget

Caches have limited space. So, when full, they must evict entries. Two classic algorithms dominate here:

Least Recently Used (LRU)

Removes the **least recently accessed** item first.

Analogy: Think of your brain's short-term memory — the least used facts fade first.

Implementation Idea:

- Use a **HashMap + Doubly Linked List**
- HashMap → quick access to nodes
- Linked List → maintains order of usage

```
[MRU] A <-> B <-> C [LRU]

Access B  => Move B to front
Evict     => Remove from tail (C)
```

Class Design:

```
class LRUCache<K, V> {
    private final int capacity;
    private Map<K, Node<K, V>> map;
    private DoublyLinkedList<K, V> dll;

    public V get(K key) { ... }
    public void put(K key, V value) { ... }
}
```

Least Frequently Used (LFU)

Removes the **least accessed** item — focuses on access frequency, not recency.

Analogy: The cafeteria stops serving dishes no one orders anymore.

Implementation Idea:

- Maintain access count for each key.
- Evict the one with the smallest frequency.

Tradeoff: LFU provides smarter eviction but is more complex and costly to maintain.

2. Write Policies — Keeping Cache & DB in Sync

When data changes, how does your cache stay consistent with the source of truth (the database)? That's where **write policies** come in.

Write-Through Cache

Every write goes to both **cache and DB** simultaneously.

- ✅ Data consistency guaranteed.
- ❌ Slower writes (since DB is also updated).

```
write() => cache.put() + db.save()
```

Used when consistency matters — like in **banking systems**.

Write-Back (Lazy Write)

Writes only to **cache first**, and updates the DB later (async or batch).

- ✅ Very fast writes
- ❌ Risk of data loss if cache crashes before sync

Used in systems where **speed > consistency** (e.g., analytics, logs).

Write-Around

Writes go **directly to DB**, skipping the cache — but reads later populate it.

- ✅ Avoids cache pollution for rarely-read data
- ❌ Read miss on fresh data

3. Pulling It All Together

A simple caching system class structure might look like this:

```
Cache
  +get(key)
  +put(key, value)
  +evict()

|
+--> LRUCache
|
+--> LFUCache
|
+--> WriteThroughCache
|
+--> WriteBackCache
```

Example: User Profile Cache

Let's imagine caching user profiles in an app like LinkedIn.

```
interface Cache<K, V> {
    V get(K key);
    void put(K key, V value);
}

class UserProfileCache extends LRUCache<String, UserProfile> {
    // integrates write policy and DB sync
}
```

Flow:

1. User opens a profile.
2. Cache hit → serve from memory.
3. Cache miss → fetch from DB → store in cache.
4. On update, use Write-Through to persist changes.

This small design drastically reduces DB load and improves latency.

Visual Snapshot

```
        ┌──────────────┐
Client →|    Cache     |— Cache Miss → DB
        └──────────────┘
              |
       Cache Hit
             ↓
         Return Data
```

Eviction:

```
LRU Order: [MRU] A → B → C [LRU]
Insert D → Evict C
```

Try It Yourself

Design a cache for your e-commerce cart system:

- Which eviction policy would you choose — LRU or LFU?
- Would you go for write-through or write-back? Why?

Implement a small LRUCache in your preferred language.

- Measure how response time improves for repeated requests.

Summary

Key Takeaways:

- Caching improves performance by reducing database hits.
- LRU and LFU are core eviction policies — balancing speed and intelligence.
- Write policies (Write-Through, Write-Back, Write-Around) determine how consistent your cache is with your DB.
- Always tailor your cache strategy to **your system's needs** — no one-size-fits-all solution.

When to Use:

- High read frequency, moderate write frequency.
- Expensive or repetitive queries.

When to Avoid:

- Highly volatile data.
- Strict consistency requirements without strong sync logic.

What's Next

Tomorrow, we'll explore **Thread Safety and Concurrency in LLD** — diving into how multiple threads safely access shared data structures like caches without race conditions.

Follow the full series here and save it for daily learning: Low-Level Design in 25 Days — A Practical Bootcamp for Devs (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)